

6. Written analysis [40 points]

Submit your answers in a PDF named pa01.pdf.

Support your conclusions using your implementation, formal reasoning, and experimental results. A returned path may differ from the sample output while still being correct if it has the same optimal cost.

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining cost it lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

Steps:

```
For i in range of current_state:
    If current_state[i] != goal_state[i]
        cost += 1
Return cost
```

The heuristic lower-bounds the minimum number of actions required to reach the goal state. This is because every legal action changes only 1 letter at a time, so any state differing from a goal in i positions needs at least i actions to fix all incorrect letters.

The real problem requires every intermediate state to be legal (within the word list provided). The heuristic I chose computes the cost without this constraint which either makes the problem cheaper to solve or equally expensive. The relaxed cost can never be greater than the true, constrained and actual cost.

Scrabble:

```
For i in range of current_state:
    If current_state[i] != goal_state[i]
        Cost += scrabble cost of goal_state[i]
Return cost
```

Every mismatched position must eventually be changed to $\text{goal}[i]$, and since scrabble cost depends solely on the inserted letter, the final correction costs exactly $\text{scrabble_value}(\text{goal}[i])$. The heuristic would sum all of the costs across the mismatched positions, lower-bounding the unavoidable scrabble cost every solution must pay.

Every solution pays at least $\text{scrabble_value}(\text{goal}[i])$ per wrong letter in order to get the final solution. Any additional detours that could be taken would only add cost since every scrabble value is greater than or equal to 1. Therefore, the true cost is always $\geq$ the heuristic's sum, meaning it never over estimates

Frequency:

```
If current_state == goal_state:
    Return 0
For i in range of current_state:
    If current_state[i] != goal_state[i]:
        Cost += 1
Return cost + dictionary[self.goal]
```

With i incorrect position, at least i actions must be made to reach the goal_state , costing exactly $1 + \text{rarity}(\text{goal})$. The other $i - 1$ actions each cost at least 1 (since every action costs $1 + \text{rarity}(w) \geq 1$). The heuristic, $i + \text{rarity}(\text{goal})$, lower-bounds the unavoidable total $(i - 1) + 1$ which is the mandatory minimum-cost plus the exact cost of the final action into the goal.

The true cost is the sum of at least i terms which are all greater than or equal to 1, with the last action equal to exactly $1 + \text{rarity}(\text{goal})$. Bounding the $i - 1$ unknown terms at their minimum and using the known exact value for the last action to the goal state gives $(i - 1) + 1 + \text{rarity}(\text{goal})$ which can only be less than or equal to the actual cost, since any detours would add more nonnegative cost.

Question 2: Does the heuristic actually help? [10 points]

Run both uniform-cost search and A* on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.9%
WHY → ASK	Scrabble	9.00	765	9.00	13	98.3%
WHY → ASK	Frequency	38.86	808	38.86	634	21.5%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.1%
BEAR → DUCK	Scrabble	11.00	2981	11.00	6	99.8%
BEAR → DUCK	Frequency	38.24	1084	38.24	302	72.1%

Calculate the percentage reduction as:

$$100 \times (\text{UCS expansions} - \text{A* expansions}) / \text{UCS expansions}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.
Yes, A* search did always return the same optimal cost. This was expected since all 3 heuristics are admissible.
2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?
For the word pairs WHY → ASK and BEAR → DUCK, A* produced the largest percentage reduction in Scrabble with 98.3% and 99.8% accordingly.
3. For each word pair, under which cost measure was the heuristic least informative?
Frequency was the least informative for both pairs by a wide margin of 21.5% for WHY → ASK and 72.1% for BEAR → DUCK, compared to the 97%+ for the other 2 cost measures in both of the cases.
4. Explain the differences using the definitions of your heuristics, not runtime alone. Different optimal paths are acceptable when their total costs are equal.
The steps and scrabble heuristics are both based directly on counting how many letters are still wrong and need to be changed. That count would decrease as the search gets closer to the goal, so it gives A* a clear sign for which nodes should actually be expanded. The frequency heuristic is $i + \text{rarity}(\text{goal})$ and the $\text{rarity}(\text{goal})$ part never changes because it's the same fixed goal added to every non-goal node no matter how close it actually is to the goal. Due to A* ranking nodes by adding the heuristic to the cost so far, adding that same fixed number ($\text{rarity}(\text{goal})$) to every node barely changes their order compared to using cost alone. This is why the frequency guided search behaves much like regular UCS and expands much more nodes than the other heuristics.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

1. State yes or no.

Step: Yes

Scrabble: No

Frequency: No

2. Justify your conclusion.

Step: Every action costs exactly 1 regardless of which letter is changed. The number of letters that are incorrect are always symmetric and it doesn't depend on which word you start from. So any path from $W1 \rightarrow W2$ can be traversed in the reverse order to get from $W2 \rightarrow W1$, using the same number of actions and therefore also the same cost.

Scrabble: The cost of an action depends only on the letter being inserted, not on what letters are being removed. Therefore the total cost of a path depends on which letters you're inserting in order to get closer to the goal. Going from $W1$ to $W2$ inserts letters that get closer to $W2$ while going from $W2$ to $W1$ inserts letters to get closer to $W1$. If the letters needed to reach one direction happen to be more costly (for example, inserting Q or Z which both have a cost of 10) than the letters needed to reach the other direction, which means the two totals won't match.

Frequency: The cost of an action is $1 + \text{rarity}(w)$, where w is the word the action produces and the initial word never contributes to the frequency cost. This means that a path's total cost doesn't include the rarity of whichever word you start from, however both include the rarity of the other word along the way including the final goal word. If you reverse the exact same path of words, the roles then would switch. The cost would then leave out the rarity of the goal and would then include the initial word instead. Unless those 2 words happen to have identical rarity values, they would never have the same cost.

3. If the answer is no, give a concrete counterexample using words from the provided dictionary.
Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Scrabble Counter-Example:

Cat $\rightarrow$ Cab

Scrabble value of b is 3

Cab $\rightarrow$ Cat

Scrabble cost of t is 1

Here you have 2 words with 1 step however as you can see $3 \neq 1$ so that costs are not the same when you try to go in reverse order.

Frequency Counter-Example:

Cat $\rightarrow$ Cot

Cost: $1 + \text{rarity}(\text{cot}) = 1 + 10.071238 = 11.071$

Cot $\rightarrow$ Cat

Cost: $1 + \text{rarity}(\text{cat}) = 1 + 6.886906 = 7.887$

Here you have different costs because the word you are substituting into $\text{rarity}()$ changes. Here as you can see, Cot and Cat both have different frequencies.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

Model: GPT-5.6 Luna

Exact Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed Heuristic:

A simple rarity-aware heuristic can be obtained by observing that **at least $h_0(n)$ actions are unavoidable**, and every action has a positive base cost of 1 plus a nonnegative rarity cost.

Let

$$r_{\min} = \min_{w \in D} \text{rarity}(w),$$

where D is the supplied dictionary. Define

$$h_R(n) = h_0(n)(1 + r_{\min})$$

or equivalently,

$$h_R(n) = \#\{i: n_i \neq g_i\}(1 + r_{\min}).$$

2. A precise formula or pseudocode for the proposed heuristic.

$R_{\min} = \min (\text{rarity}(w) \text{ for } w \text{ in dictionary } D)$

$h_R(n) = h_0(n) \times (1 + r_{\min})$

Where $h_0(n)$ is the number of positions where the state (n) differs from the given goal (the baseline heuristic is given within the prompt).

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

Claim: $h_R(n) = h_0(n) \times (1 + r_{\min})$ is an admissible heuristic

Proof:

Let $a = h_0(n)$. Each action changes only 1 letter which means that at least k actions are needed to reach the goal. Every action costs $1 + \text{rarity}(w)$ for whatever word w ends up producing. Plus, due to $r_{\min}$ being the dictionary-wide minimum rarity, every action costs at least $1 + r_{\min}$. Therefore the true remaining cost is at least $k \times (1 + r_{\min}) = h_R(n)$. Because $h_R(n)$ never exceeds the actual cost, it is admissible.

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.

Surprisingly, the heuristic proposed by GPT-5.6 Luna was correct so there is no need to make a small repair or even replace it with a safe alternative.

I checked the dictionary of words and $r_{\min} = 0.016$ and 0 words have a rarity of exactly 0, confirming h_R is strictly more informative than h_0 for this dictionary

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Word Pair	Heuristic	Solution Cost	Nodes Expanded
WHY → ASK	h_0 (baseline)	38.86	755
WHY → ASK	h_R (rarity known)	38.86	754
BEAR → DUCK	h_0 (baseline)	38.24	720

BEAR → DUCK	h_R (rarity known)	38.24	714
-------------	--------------------	-------	-----

Both of the heuristics used found the same optimal costs but the reduction in the nodes expanded is not substantial. While h_R is technically admissible and technically more informative than h_0 , the practical benefit is actually minimal given how small $r_{\min}$ is relative to the range of rarity values in the dictionary.

I checked the dictionary of words and $r_{\min} = 0.016$ and 0 words have a rarity of exactly 0, confirming h_r is strictly more informative than h_0 for this dictionary.